Fundamentos da arquitetura back-end e papel do servidor

Objetivos da aula

- Nesta aula vamos compreender o papel do back-end em uma aplicação web moderna. Ao final da aula você será capaz de:
  - Entender o que é arquitetura back-end
  - Explicar o papel do servidor de aplicação
  - Identificar responsabilidades da camada back-end
  - Compreender o fluxo de uma requisição HTTP
  - Relacionar front-end, back-end e banco de dados
  - Reconhecer a importância da separação de responsabilidades

O que é Back-end?

- O back-end é a camada responsável pelo processamento interno do sistema.
- É onde as regras são aplicadas, os dados são validados e as decisões são tomadas.
- O usuário não vê o back-end, mas depende completamente dele.

O que é Back-end?

- Back-end é a camada responsável por:
  - Processar regras de negócio
  - Manipular dados
  - Garantir segurança
  - Integrar sistemas
  - Responder requisições do cliente

É a “parte invisível” da aplicação para o usuário final.

Visão Geral de uma Aplicação Web

- Uma aplicação web moderna geralmente é composta por:
  - Front-end (interface do usuário)
  - Back-end (processamento e regras)
  - Banco de dados (persistência)

Arquitetura típica em três camadas.

Arquitetura Cliente-Servidor

- Modelo fundamental da web:
  - Cliente envia requisição
  - Servidor processa
  - Servidor envia resposta

Baseado no protocolo HTTP (Hypertext Transfer Protocol).

O Papel do Servidor

- O servidor é o ambiente onde o código do back-end é executado.
- Ele recebe requisições, executa regras, consulta dados e retorna respostas estruturadas ao cliente.
- Sem o servidor, não há processamento.

O Papel do Servidor

- O servidor é responsável por:
  - Receber requisições HTTP
  - Interpretar a rota solicitada
  - Executar regras de negócio
  - Consultar banco de dados
  - Retornar resposta estruturada (JSON, HTML etc.)

O que é um Servidor de Aplicação?

- O servidor de aplicação organiza o fluxo das requisições e executa o código do sistema.
- Frameworks como Spring Boot, Django e Express.js fornecem a estrutura necessária para isso.
- Eles abstraem detalhes técnicos e permitem foco na regra de negócio.

O que é um Servidor de Aplicação?

- Servidor de aplicação:
  - Executa código do back-end
  - Gerencia sessões
  - Controla autenticação
  - Orquestra requisições

Exemplos populares: Spring Boot, Django e Express.js.

Requisições HTTP

- Toda interação começa com uma requisição HTTP.
- Ela possui um método, uma URL e, em alguns casos, um corpo com dados.
- O servidor interpreta essa requisição para decidir o que fazer.

Requisições HTTP

- Principais métodos:
  - GET → Buscar dados
  - POST → Criar dados
  - PUT → Atualizar dados
  - DELETE → Remover dados

Cada requisição gera uma resposta.

Estrutura de uma Requisição HTTP

- Uma requisição contém:
  - Método
  - URL
  - Cabeçalhos (headers)
  - Corpo (body)

Esses elementos orientam o processamento no servidor.

Resposta HTTP

- Após o processamento, o servidor retorna uma resposta.
- Ela contém um código de status e, normalmente, dados em formato JSON.
- Essa resposta será interpretada pelo front-end.

Estrutura da Resposta HTTP

- Uma resposta contém:
  - Código de status (200, 404, 500 etc.)
  - Cabeçalhos
  - Corpo da resposta

O back-end define o que será retornado.

Lógica de Negócio

- A lógica de negócio representa as regras do sistema.
- Ela define comportamentos, restrições e decisões.
- Exemplo: validar saldo antes de permitir uma compra.
- Essa lógica deve estar no back-end.

Por que a Lógica Fica no Back-end

- Se regras importantes fossem implementadas no front-end, poderiam ser facilmente burladas.
- Centralizar decisões no back-end garante segurança, consistência e controle.

Lógica de Negócio

- A lógica de negócio:
  - Define regras do sistema
  - Impõe validações
  - Controla fluxos
  - Garante consistência

Exemplo: não permitir saldo negativo.

Persistência de Dados

- O back-end:
  - Armazena dados
  - Recupera dados
  - Atualiza registros
  - Garante integridade

Interage com bancos relacionais e não relacionais.

Banco de Dados na Arquitetura

- Tipos comuns:
  - Relacionais (SQL)
  - Não relacionais (NoSQL)

Exemplos: PostgreSQL, MySQL e MongoDB.

Comunicação entre Camadas

- Fluxo comum:
  - Front-end → Back-end → Banco de dados → Back-end → Front-end
- Cada camada tem responsabilidades específicas.

Separação de Responsabilidades

- Separar responsabilidades significa que cada parte do sistema faz apenas o que lhe compete.
- Isso reduz acoplamento, melhora manutenção e facilita testes.
- É um princípio central da boa arquitetura.

Separação de Responsabilidades

Princípio fundamental:

- Cada camada deve ter uma função clara.
- Benefícios:
  - Manutenção facilitada
  - Testabilidade
  - Escalabilidade
  - Organização do código

Arquitetura em Camadas

- Uma forma comum de organizar o back-end é por camadas.
- Cada camada tem um papel específico e colabora com as demais para cumprir o fluxo completo da requisição.

Arquitetura em Camadas

- Camadas comuns:
  - Controller: É a porta de entrada da aplicação.
  - Service: Contém a lógica de negócio.
  - Repository: Responsável pelo acesso ao banco de dados.
  - Database: Camada final do fluxo.

- Cada uma resolve um problema específico.

Controller

- Responsável por:
  - Receber requisições
  - Validar entrada básica
  - Delegar para a camada de serviço
  - Retornar resposta

Não deve conter regra de negócio complexa.

Service

- Responsável por:
  - Implementar lógica de negócio
  - Aplicar regras
  - Coordenar operações

É o “cérebro” do sistema.

Repository

- Responsável por:
  - Acessar banco de dados
  - Executar consultas
  - Mapear dados

Abstrai a persistência.

Integração com Serviços Externos

- O back-end também pode se comunicar com sistemas externos.
- Por exemplo, serviços de pagamento como Stripe ou autenticação via Google.
- Ele atua como intermediador dessas integrações.

Integração com Serviços Externos

- Back-end pode consumir:
  - APIs externas
  - Sistemas de pagamento
  - Serviços de autenticação

Exemplo: Stripe, Google.

APIs e Back-end

- API (Application Programming Interface):
  - Define como sistemas se comunicam
  - Geralmente usa REST
  - Troca dados em JSON

O back-end expõe APIs para o front-end.

APIs e Back-end

- O back-end geralmente expõe APIs.
- Uma API define como sistemas se comunicam e quais dados podem ser acessados ou modificados.
- O padrão mais comum é o REST.

REST e Arquitetura Web

- REST organiza a aplicação em recursos acessíveis via URL.
- Cada recurso pode ser manipulado usando métodos HTTP.
- É simples, escalável e amplamente adotado.

REST e Arquitetura Web

- REST é baseado em:
  - Recursos
  - URLs
  - Métodos HTTP
  - Stateless

Amplamente adotado em aplicações modernas.

Stateless e Escalabilidade

- Em arquiteturas REST, cada requisição é independente.
- O servidor não depende de informações salvas entre requisições.
- Isso facilita escalabilidade.

Stateless e Escalabilidade

- Stateless significa:
  - Cada requisição é independente
  - Servidor não mantém estado entre requisições

Facilita escalabilidade horizontal.

Segurança no Back-end

- O back-end é responsável por autenticar usuários, autorizar ações e proteger dados sensíveis.
- Ele funciona como a barreira de segurança do sistema.

Segurança no Back-end

- Responsabilidades:
  - Autenticação
  - Autorização
  - Criptografia
  - Proteção contra ataques

Back-end é guardião da segurança.

Autenticação e Autorização

- Autenticação → Quem é o usuário?
- Autorização → O que ele pode fazer?

Exemplos: JWT, OAuth.

Tratamento de Erros

- Um bom back-end trata erros de forma controlada.
- Ele retorna códigos HTTP adequados e evita expor detalhes internos da aplicação.

Tratamento de Erros

- Back-end deve:
  - Capturar exceções
  - Retornar códigos HTTP adequados
  - Não expor detalhes internos

Exemplo: 500 para erro interno.

Logs e Monitoramento

- Sistemas profissionais registram eventos e erros.
- Ferramentas como Prometheus e Grafana ajudam a acompanhar o comportamento da aplicação.

Logs e Monitoramento

- Importância:
  - Rastrear erros
  - Analisar performance
  - Auditar ações

Ferramentas comuns: Grafana, Prometheus.

Escalabilidade

- Back-ends devem ser projetados para crescer.
- Isso pode ocorrer aumentando recursos da máquina ou distribuindo a aplicação em múltiplas instâncias.

Escalabilidade

- Tipos:
  - Vertical (aumentar máquina)
  - Horizontal (aumentar instâncias)

Back-end deve ser projetado para crescer.

Arquitetura Monolítica

- No modelo monolítico, toda a aplicação é executada como um único bloco.
- É simples no início, mas pode se tornar difícil de escalar.

Arquitetura Monolítica

- Características:
  - Um único projeto
  - Deploy único
  - Simplicidade inicial

Desvantagem: difícil escalar partes isoladas.

Arquitetura de Microserviços

- Em microserviços, o sistema é dividido em serviços independentes.
- Cada serviço possui responsabilidade própria e pode ser implantado separadamente.
- Ferramentas como Kubernetes auxiliam na orquestração.

Arquitetura de Microserviços

- Características:
  - Serviços independentes
  - Deploy separado
  - Comunicação via API

Exemplo de orquestração: Kubernetes

Comunicação entre Serviços

- Microserviços podem se comunicar via HTTP ou mensageria.
- Soluções como RabbitMQ e Apache Kafka são comuns nesse cenário.

Comunicação entre Serviços

- Pode ocorrer via:
  - HTTP
  - Mensageria
  - Eventos

Exemplos: RabbitMQ, Apache Kafka, Amazon SQS.

Back-end e Performance

- O back-end deve responder rapidamente às requisições.
- Consultas eficientes, uso adequado de recursos e boas práticas arquiteturais impactam diretamente na experiência do usuário.

Back-end e Performance

- Aspectos importantes:
  - Tempo de resposta
  - Uso eficiente de recursos
  - Caching Consultas otimizadas

Back-end e Manutenibilidade

- Um back-end bem estruturado é fácil de entender, modificar e testar.
- Boa arquitetura reduz o custo de evolução do sistema.

Back-end e Manutenibilidade

- Boas práticas:
  - Código limpo
  - Baixo acoplamento
  - Alta coesão
  - Testabilidade

Relaciona-se diretamente à qualidade do software.

Fluxo Completo de uma Requisição

- 1. Usuário clica no botão
- 2. Front-end envia requisição
- 3. Back-end processa
- 4. Consulta banco
- 5. Retorna resposta
- 6. Interface é atualizada

Fluxo essencial de qualquer sistema web.

Síntese da Arquitetura Back-end

- O back-end processa regras, protege dados, integra sistemas e fornece APIs.
- Ele é o núcleo lógico da aplicação web.

Síntese da Arquitetura Back-end

- O back-end é responsável por:
  - Processar regras
  - Proteger dados
  - Integrar sistemas
  - Garantir consistência
  - Fornecer APIs

Encerramento

- Você aprendeu:
  - O que é back-end
  - O papel do servidor de aplicação
  - Como funciona uma requisição HTTP
  - A importância da lógica de negócio
  - A relação entre front-end, back-end e banco
  - O conceito de separação de responsabilidades
  - Arquiteturas monolítica e de microserviços

PUC Minas

© **PUC Minas** • Todos os direitos reservados, de acordo com o art. 184 do Código Penal e com a lei 9.610 de 19 de fevereiro de 1998.
Proibidas a reprodução, a distribuição, a difusão, a execução pública, a locação e quaisquer outras
modalidades de utilização sem a devida autorização da Pontifícia Universidade Católica de Minas Gerais.